Bluebell and X-CSL lights updater for XP12

Inhaltsverzeichnis

Files in this package

```
├── configs
│   ├── aircrafts.json
│   ├── config.ini
│   ├── light_params.json
│   └── _version.py
├── decorators
│   └── time_benchmark.py
├── Documentation
│   ├── gpl-3.0.txt
│   └── Manual.pdf
├── helpers
│   ├── aircraft_helpers.py
│   ├── aircraft_light_params.py
│   ├── custom_exceptions.py
│   ├── helpers.py
│   └── __init__.py
└── lights_updater.py
```

Quickstart tl;dr

1. Unzip the file to a place of choice
2. Edit the configs/config.ini and set the csl_path value.
3. Start the tool with ./lights_updater.py

OR alternative

3. If you don't want to use the path in the config, or you only want to test the tool, or overrule the setting:
4. Start the tool with ./lights_updater -p D:\path\to\cls\files\diretory

   (or the --path argument instead whichever you prever)

Notice!

The path must contain at least one xsb_aircraft.txt as this is the base from where the tool creates the paths to the aircraft object files. It's the same as LiveTraffic and I believe also xPilot do.

What is it?

I use LiveTraffic and I really like it. But! When I am at the holding point of a runway and I look if someone is on final, I don't see anything at daylight or the plane must be that close that you can see its body. So a (fast) plane on final can be missed. Yes, sure, you can use ATC, but I do not do that for every flight. Even planespotting is less nice if you cannot spot the planes in line as they are closing to the runway. See the screenshots.
I am not patient enough to change all the objects by hand to convert them to the new XP12 light params. That was the reason to build this tool. And I learned more about programming in Python. Double win!
X-CSL and Bluebell still have the *"old"* XP11 lightparams in the aircraft objects. This tool changes them to XP12 params.
I change only the "necessary" lights: Landing lights, taxi lights, nav lights, beacon and strobe lights. All other lights are untouched.

Installation:

!! First of all !!

!! Make a backup of your Livetraffic Resourcesdirectrory !!

1. It's always a good habit to make a backup of things you are going to change.
2. Copy the zip to destination of your choice
3. Unzip the file
4. Edit and set the path to your CSL resources directory in the config ini.

```
[csl]
; Example: csl_path = /path/to/XP12/plugins/directory/LiveTraffic/Resources/CSL
csl_path = D:/path/to/your/X-Plane 12/Resources/plugins/LiveTraffic/Resources/CSL
```

Example of the csl part of the config.ini

Notice!

The path must contain at least one xsb_aircraft.txt as this is the base from where the tool creates the paths to the aircraft object files. It's the same as LiveTraffic and I believe also xPilot do.

5. Run it. See for more info on the next page!

How does it work?

This tool works in four steps.

- First it makes a backup of the existing object files.
- Then it deletes the xpmp2 copies, Livetraffic made. Not doing this led to a strange behavior because LiveTraffic uses those files to correct some other shortcomings of older aircraft objects. It therefore is also a kind of “cache”. Sometimes I couldn’t see the changes I made (Still landing lights at daylight after rollback for instance).
- Then it reads the original file, changes the light params and writes those to a temporary file.
- If all is ready, the temporary files get copied over the original ones.
- Done!

How long does it take to convert?

The SSD with X-Plane 12 on it:
lights_updater: INFO - Processing done, 1798 files have been processed!
lights_updater: INFO - It took 24.00 seconds!

An old HDD with both X-CSL and Bluebell packages:
lights_updater: INFO - Processing done, 1798 files have been processed!
lights_updater: INFO - It took 36.00 seconds!

Your millage will vary. Hardware, OS and a lot more possibilities :-)

How to:

Just start it with either:

```
python lights_updater.py
Or
./lights_updater.py
Or
python.exe lights_updater.py
```

Normal start

No arguments! But the path to the cls file must be set in the config.ini!

Set path with argument

You can specify a path besides the setting in the config.ini. To do so add the **-p** or **--path** and the path to your csl file. For example: ./lights-updater.py -p D:\X-Plane 12\Resources\etc\etc

Undo

This tool makes a backup of each object file that is in the xsb_aircrafts.txt, so there is an option to undo the changes. To do so, start the tool with **-u,** or **--undo** option.

Remove backup files

If all is good, there is a option to delete the backups after all is fine. This can be done by starting the tool with the **-r** or --**remove-backups** option. But be careful. Gone is gone. At least on Linux.

Redo (with new params)?

You can restart the tool if you change values in the light_params.json. In the older version(s) you had to first run **undo**, now running the tool again will rebuild the object files with the new params.

Show usage

The usage can be viewed when you start the tool with the **-h** or --**help** option.

```
lights_updater
usage: lights_updater.py [-h] [-p csl_path] [-u] [-r]

This program can convert XP11 lightparams of CSL aircraft objetcs to the new XP12
specifications.
Developed for getting landing lights for LifeTraffic.
Works only partial with custom CSL aircraft.

For normal start you don't need any arguments.
But then you MUST specify the csl_path in the config.ini file!

Optional arguments:
  -h, --help            show this help message and exit
  -p csl_path, --path csl_path
                        Set path to location of CSL aircrafts.
                        Be aware that if there are whitespaces in the path, you MUST use
quotation marks around the CSL_PATH!
                        To be save, always use them.
  -u, --undo            Rolls back the previous made changes!
  -r, --remove-backups  Removes the backup files. Be careful!

Attention!!
If you specify a path with -p / --path when processing the objects and you want to
undo your changes,
or remove the backup files with -r / --remove-backups, you need the specify the path
again!
If you don't and there is a path specified in the config.ini, results may not be what you
expected!

Now you know!
```

Image 1: Overview of usage

Tested on:

- Linux Mint 21 with Python 3.10
- Windows 10 Pro with Python 3.12
- Bluebell packages
- X-CSL packages

Some screens to show the result:

Yes, I know, these screenshots are small. A4 size problems…

Specials

aircrafts.json

This file contains the aircraft's by type as far as I could determine the correct type. I used the ICAO website a lot for codes unknown to me. [Search by: Aircraft Type Designators]
If you detect an aircraft not being converted correctly or is missing completely, you can add it in here.

light_params.json

This file contains the lightparams per aircraft category. Those are partially copied from airplanes that are XP12 compatible, partially an estimation by me. Here you can change the params for each light.

Typo’s:

As English is not my mother tongue I am sure that not all is correct English :-) I did my best at least.
Those who find typo’s or other errors may keep them as a free gift from me :-)

Custom CSL aircraft:

I did some tests. Not all were successful.

Sometimes the xsb_aircraft.txt is not correct filled with the aircraft data (case-sensitive filenames for those with a Microso.. mindset) or aircraft are completely missing. So some will work, some don't. Try if you dare :-) The rate of success could be higher on Windows.
As long as the xsb_aircraft.txt is correct

AND

there are no strange combinations of lightparams (I found the new specifier LIGHT_PARAM with the old light parameters), the lights will be converted.

In case of minor errors:

Just in case there are some errors because of not compatible objects:
You can override stopping on error with the setting in the config.ini. Just set it to False. Then as long as there are no fatal errors the tool will continue to convert.

License

Just for the completeness.